

GRAPE: A unified deep learning framework for robust combinatorial optimization

Yingze Zhong^a, Hui Li^{a,*}, Qinghang Zhai^a, Zhan Shi^a and Jianyong Sun^a

^a*School of Mathematics and Statistics, Xi'an Jiaotong University, Xi'an, China*

ARTICLE INFO

Keywords:

Robust optimization
Combinatorial optimization
Deep learning
Reinforcement learning

ABSTRACT

Robust combinatorial optimization (RCO) aims to obtain high-quality solutions under uncertainty in real-world problems, while classical techniques depend on problem-specific modeling and costly nested computations, which limit reusability and efficiency. In this paper, we present GRAPE, a unified deep learning framework for RCO that integrates solution generation, robustness estimation, and adaptive adjustment into an end-to-end learning pipeline. By training on instance distributions, our framework can jointly address min-max, regret-based, k -adaptability, and recoverable RCO formulations within a single modular architecture, eliminating the need for specialized solvers and replacing costly worst-case evaluations with learned surrogate models. Experiments on robust traveling salesman problems demonstrate significant reductions in computational cost and solid generalization across different uncertainty budgets and problem scales. This work bridges learning-to-optimize and robust optimization, providing a modular, scalable and generalizable paradigm for complex decision-making under uncertainty.

1. Introduction

Combinatorial optimization (CO) problems have attracted increasing interest from the machine learning community due to their simple structure yet high computational complexity (typically NP-hard). In recent years, a variety of deep reinforcement learning (DRL) methods have emerged to address these problems [1–3]. In classical CO settings, the cost coefficients in the objective function are assumed to be fixed and precisely known. However, this rarely holds in real-world applications, where uncertainty is prevalent. Taking the traveling salesman problem (TSP) as an example, factors such as traffic situations, weather conditions, and fuel consumption can significantly affect the travel cost between cities. Deviations between actual and nominal costs can degrade the performance of these idealized models, leading to suboptimal solutions. Such challenges have motivated research into robust combinatorial optimization (RCO), which seeks solutions that remain effective under worst-case scenarios.

However, existing RCO approaches still rely heavily on exact or heuristic solvers, which require problem-specific modeling and handcrafted algorithm design [4]. These methods are often hard to generalize across different uncertainty formulations and typically suffer from computational challenges due to nested bilevel structures.

In parallel, deep learning (DL) has achieved notable progress in classical CO problems by learning solution construction policies in an end-to-end, data-driven manner. Pioneering works such as Pointer Networks [5], Reinforcement Learning (RL)-based Model [6], Attention Model [7], and Matnet [8] have shown strong performance on tasks like TSP

and VRP. However, applying DL to RCO remains underexplored, largely due to the difficulty of handling uncertainty during training.

Neural surrogate models have also been widely studied to approximate complex objective functions and accelerate mixed-integer program solving [9], offering a means to amortize expensive computation. However, such surrogate-based methods are still rare in the robust optimization. A notable exception is Neur2RO [10], which trains neural approximators to mimic worst-case objectives under discrete scenario sets. In addition, other machine learning techniques have recently been applied to enhance traditional RCO methods [11, 12].

To address these limitations, we propose **GRAPE** (Generator, Robustness Estimator, Adjuster, and Picker under uncertain Environments), a unified deep learning framework for robust combinatorial optimization. Inspired by the structured clustering of grapes, the framework decomposes the solution process into modular, learnable components and integrates them into an end-to-end pipeline that supports both solution generation and uncertainty-aware refinement. Its architecture is designed to flexibly accommodate a wide range of robust formulations by approximating worst-case reasoning and second-stage adaptation through neural surrogates.

In this work, we evaluate **GRAPE** on four representative RCO formulations (see Table 1), namely min-max, min-max regret, k -adaptability, and recoverable formulation, using the robust traveling salesman problem (RTSP) as a benchmark. We adopt the widely-used budgeted uncertainty set to model cost perturbations, and assess the framework's generalization under varying uncertainty levels. Experimental results demonstrate that our method consistently outperforms baselines in solution quality, computational efficiency, and generalization capacity.

Our main contributions are as follows:

*Corresponding author

✉ lihui10@xjtu.edu.cn (H. Li)

ORCID(s): 0000-0002-5357-2438 (H. Li)

Table 1
Representative formulations of Robust Combinatorial Optimization.

Formulation	Objective	Description
Min-max	$\min_{x \in \mathcal{X}} \max_{c \in \mathcal{U}} f(x, c)$	Minimize the worst-case cost.
Min-max regret	$\min_{x \in \mathcal{X}} \max_{c \in \mathcal{U}} \left[f(x, c) - \min_{x^* \in \mathcal{X}} f(x^*, c) \right]$	Minimize deviation from optimal per scenario.
k -adaptability	$\min_{\{x_1, \dots, x_k\} \subseteq \mathcal{X}} \max_{c \in \mathcal{U}} \min_{i=1, \dots, k} f(x_i, c)$	Pre-select k options, pick best after realization.
Recoverable	$\min_{x \in \mathcal{X}} \max_{c \in \mathcal{U}} \min_{x' \in \mathcal{N}(x)} f(x', c)$	Allow limited post-adjustment after realization.

- We propose **GRAPE**, the first unified and end-to-end deep reinforcement learning framework for robust combinatorial optimization that adopts a modular design.
- We design a novel robustness estimation module that efficiently approximates the worst-case objective under uncertainty, enabling fast and accurate reward evaluation and eliminating costly nested min-max optimization.
- We introduce an adaptive adjustment module that learns to generate second-stage solutions, enabling support for both k -adaptability and recoverable optimization paradigms.

The remainder of this paper is organized as follows. Section 2 introduces the foundations of robust optimization and its combinatorial variants. Section 3 reviews related work on RCO, deep learning for CO, and learning-based surrogate models. Section 4 presents the proposed GRAPE framework in detail. Section 5 reports experimental results on several robust routing tasks. Section 6 concludes the paper and discusses future research directions.

2. Background

Robust optimization (RO) serves as an optimization paradigm for decision-making under uncertainty. Rather than relying on probabilistic assumptions, RO seeks the worst-case optimal solution that guarantees reliable performance across all realizations within a given uncertainty set.

Formally, the general RO problem can be stated as:

$$\min_{x \in \mathcal{X}} \max_{\xi \in \mathcal{U}} f(x, \xi) \quad (1)$$

where $x \in \mathcal{X}$ denotes the decision variable and $\xi \in \mathcal{U}$ represents the uncertain parameter. The inner maximization identifies the worst-case realization, while the outer minimization seeks its corresponding robust decision.

In discrete and combinatorial settings, this paradigm leads to RCO problems, which are particularly challenging due to the combinatorial structure of the decision space and the high dimensionality of the uncertainty set.

Section 2.1 introduces the formulation of RCO, while Section 2.2 introduces the budget uncertainty set, a widely used model that balances robustness and tractability.

2.1. Formulation of RCO

RCO problems can be categorized into several formulations based on how decision-making under uncertainty is modeled (see Table 1).

The classical min-max formulation seeks a single solution that minimizes the worst-case cost. The min-max regret formulation, in contrast, aims to minimize the largest gap between the chosen solution and the optimal one under each scenario, thereby balancing performance across different outcomes [13, 14]. The k -adaptability formulation introduces limited flexibility by selecting k candidate solutions in advance, from which the best is chosen after the uncertainty is realized [15, 16]. Lastly, the recoverable formulation allows for limited modification of a nominal solution once the actual scenario is known, capturing realistic settings where partial adjustments are feasible [17–20].

2.2. Budget uncertainty set

In RCO, the uncertainty set $\mathcal{U}$ specifies how cost values may vary under uncertainty. The simplest choice is the box uncertainty set, where each cost varies independently within a fixed interval, but this often leads to overly conservative solutions. To mitigate this, the budget uncertainty set [21] restricts total allowable deviation via a budget parameter Γ , offering a more practical balance between robustness and optimality.

The budget uncertainty set is defined as:

$$\mathcal{U} = \left\{ c \mid c_{ij} = \underline{c}_{ij} + \xi_{ij}(\bar{c}_{ij} - \underline{c}_{ij}), \xi_{ij} \in [0, 1], \sum \xi_{ij} \leq \Gamma \right\}.$$

Here, c is the uncertain cost matrix, with lower bound $\underline{c}_{ij}$ (nominal cost) and upper bound $\bar{c}_{ij}$ (worst-case cost). The parameter Γ limits the total deviation across all components. Note that when $\Gamma = 0$, the model reduces to the nominal problem, and when $\Gamma \geq n(n-1)$, it defaults to a box uncertainty problem. Both of them correspond to deterministic settings. Notably, for certain problems like the RTSP, the threshold Γ that leads to deterministic cost can be as small as n . We prove this in *Appendix C*.

Other uncertainty models such as ellipsoidal sets are mainly suited to continuous or convex optimization problems. Since RCO involves discrete decision spaces, the budget set is generally preferred in the literature for both modeling fidelity and computational tractability, which is why we adopt it in our experiments.

3. Related work

This section reviews three lines of research relevant to our study: (i) RCO, including classical formulations and solution methods; (ii) deep learning approaches for deterministic CO; and (iii) learning-based surrogate models used to approximate optimization components.

3.1. RCO

As mentioned before, RCO extends RO to discrete domains. Prior studies span several classical formulations and propose methods ranging from exact algorithms to heuristics and metaheuristics. Early works focused on extending the foundations of RO [21, 22] to discrete settings, leading to exact algorithms based on scenario enumeration, dualization, and decomposition. For specific problems such as the RTSP, interval- or budget-based uncertainty sets have enabled specialized branch-and-cut methods [23].

Beyond exact solvers, substantial researches have explored heuristic and metaheuristic approaches, including evolutionary strategies [24, 25], local search and iterated improvement under uncertainty [20], and multi-objective viewpoints that treat robustness as a competing objective [26]. These methods aim to balance solution computational tractability and scalability when exact robust optimization becomes infeasible.

Despite these advances, most existing RCO methods still rely heavily on problem-specific models or repeated inner-loop optimization, which limits their scalability and generality across broader CO problems. This motivates the development of learning-based frameworks such as GRAPE.

3.2. DL for CO

In the past decade, significant progress has been made in applying DL to CO, often referred to as Neural Combinatorial Optimization (NCO). Pointer Network [5] first employed a sequence-to-sequence model to select nodes sequentially based on the current partial solutions. Building on this, Bello et al. [1] incorporated reinforcement learning into this framework by training the model with policy gradients, demonstrating the feasibility of learning combinatorial heuristics without supervision. Nazari et al. [6] further extended this idea to the vehicle routing problem (VRP), showing that RL can generalize across instances of varying sizes and constraints. Later, Kool et al. [7] improved this paradigm with attention-based architectures and policy gradient training, while Kwon et al. [27] proposed POMO, a multi-start policy optimization strategy capable of generating multiple high-quality tours in parallel. More recent developments, such as UniCO [28], BOPO [29], and URS [30], continue to expand the applicability and generalization ability of neural solvers across diverse routing tasks.

Although these approaches demonstrate strong generalization and scalability, they primarily focus on deterministic settings and largely overlook uncertainty, which is prevalent in many real-world CO applications. Xiao et al. [14] address uncertainty by training a regret-based neural routing solver, but the approach relies on architectural encoding of

perturbations and remains specific to the regret formulation. These limitations indicate that more flexible and efficient mechanisms are required to reason about uncertainty. This naturally leads to learning-based surrogate models, which we discuss next.

3.3. Learning-based surrogate models in optimization

In many optimization settings, computing the exact objective or solving intermediate subproblem scan be computationally expensive, especially in robust formulations involving nested or repeated optimization. Learning-based surrogate models provide an effective means to amortize this cost.

One major category is objective function approximation, where a neural network predicts the value of a complex or intractable objective. Such surrogates have been widely used in multi-objective optimization [31], Bayesian optimization [32] and bilevel optimization [33], enabling fast evaluation and supporting gradient-free search or learning procedures. In combinatorial optimization, objective approximators further assist learning-based solvers by providing rapid feedback for candidate evaluation and policy improvement.

The second category involves optimization-operator surrogates, where neural networks replace or approximate an explicit optimization operation within a larger algorithmic pipeline. A representative example is Neur2RO [10], which integrates a learned surrogate into the master-subproblem iterations of a MILP-based column-and-constraint generation (CCG) framework for fully recoverable two-stage robust optimization. Although effective, this approach requires repeated per-scenario MILP solves to maintain feasibility, leading to substantial computational overhead and limiting scalability for large discrete problems.

These developments highlight both the potential and the limitations of existing surrogate designs. Together, they motivate the need for a unified and scalable surrogate-guided framework for RCO that supports diverse robust formulations without architecture-specific uncertainty encodings or repeated exact inner optimization. This is precisely the focus of GRAPE.

4. The unified framework for RCO (GRAPE)

4.1. Framework overview

The GRAPE framework addresses robust combinatorial optimization by decomposing the learning and decision-making process into four interacting modules: Solution Generator, Robustness Estimator, Solution Adjuster, and Solution Picker. As illustrated in Figure 1, the framework begins with an uncertain RCO instance as input, and processes it through a sequence of neural components that capture the structure of robust reasoning. An RCO instance consists of a nominal CO problem, an uncertainty set $\mathcal{U}$ describing perturbations, and a robust formulation that specifies how decisions respond to uncertainty.

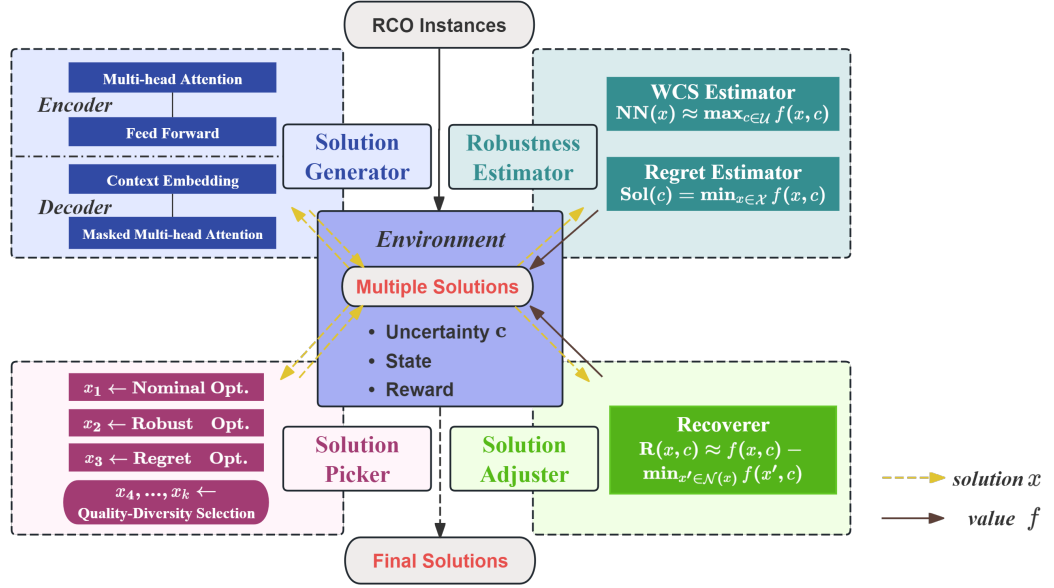


Figure 1: The framework of GRAPE for RCO.

Depending on the specific robust formulation, GRAPE dynamically utilizes different modules to approximate worst-case objectives, refine candidate solutions, and select final robust decisions. These modules are unified under a shared environment that maintains the uncertainty representation, the current state of the solution process, and the reward signals used for learning. This environment provides the interface through which multiple generated solutions interact with perturbed cost realizations, enabling reinforcement learning to incorporate uncertainty-aware feedback throughout the decision pipeline.

In the following subsections, we describe each component in detail, using the RTSP as the primary case study due to its representative combinatorial structure and rich uncertainty models.

Although RTSP serves as our example, GRAPE is designed as a unified framework applicable to a wide range of RCO problems. Adapting the framework to another combinatorial domain only requires replacing the Generator with a task-specific solver, whereas the Estimator, Adjuster, and Picker remain formulation-driven and problem-agnostic.

4.2. Solution Generator

As solving the nominal problem (e.g., solving TSP for RTSP variants) is essential, the Solution Generator module serves as the foundational module of GRAPE. It is model-agnostic, allowing integration of various DRL-based CO solvers. This design ensures flexibility across problem domains and supports the direct incorporation of state-of-the-art solution generators as plug-in components.

In the case of robust routing problems such as the RTSP, we instantiate the Generator module based on the POMO framework [27], leveraging its attention-based architecture

and multi-start decoding strategy. The architecture of the Generator module is illustrated in Figure 2. This design offers several advantages in the context of RCO:

- It produces a diverse set of candidate solutions that can support subsequent components, including robustness estimation and adaptive selection;
- It naturally aligns with the robustness objective, as solution diversity serves as a hedge against worst-case scenarios;
- It facilitates two-stage decision-making by readily providing multiple first-stage candidates in a single forward pass.

Importantly, the Generator module focuses exclusively on deterministic information, namely the nominal cost matrix $\underline{c}$, and does not incorporate uncertainty parameters such as the deviation matrix $\bar{c} - \underline{c}$ or the uncertainty budget Γ as inputs. This design choice serves two critical purposes in the framework:

- First, by restricting the Generator's input to deterministic data, we prevent early exposure of uncertainty information that could cause the model to overfit to specific perturbation patterns. This helps the model generalize across varying uncertainty scales and budgets, adapting to different levels and extents of perturbations.
- Second, by deferring uncertainty and robustness modeling to the training process, the framework simplifies the Generator's design and promotes modularity by

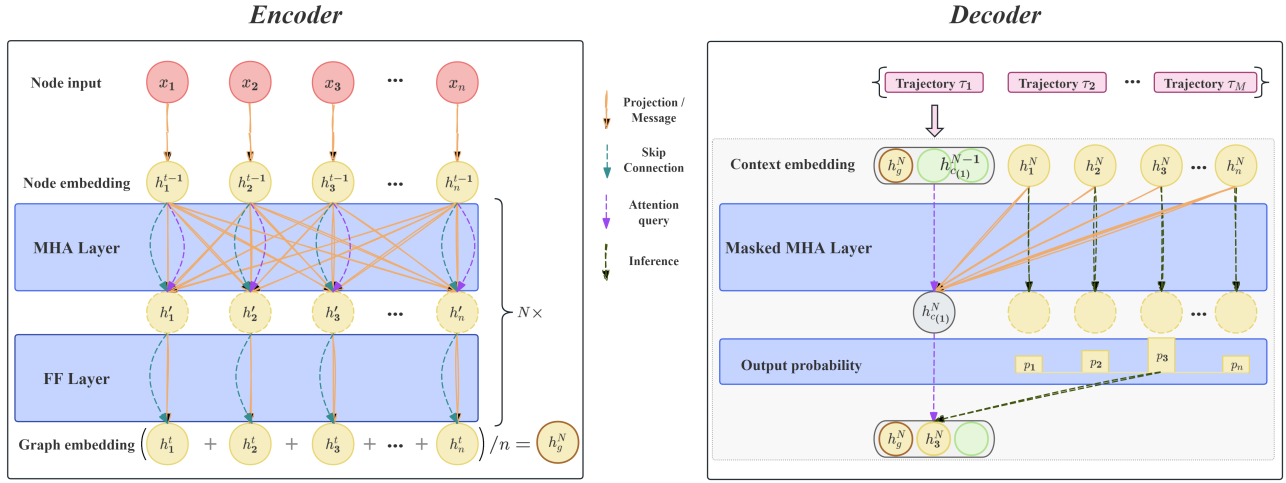


Figure 2: Illustration of the Generator module, including the Encoder, Decoder, and multi-trajectory decoding strategy.

decoupling uncertainty reasoning into dedicated components. This separation also helps improve training efficiency by localizing complex robustness computations within the Estimator and Adjuster modules.

To instantiate the Generator, we adopt the standard encoder-decoder architecture widely used in neural routing models [7]. The encoder transforms problem-specific node features (e.g., coordinates in RTSP) into contextual embeddings through a stack of attention layers, allowing the model to capture the structural dependencies within the CO instance.

The decoder then generates tours autoregressively. At each step, the model chooses the next node based on attention over the encoder embeddings and the current partial tour state. This mechanism allows the policy to generate a feasible tour while maintaining permutation invariance.

A key property of the POMO-based design is its multi-start decoding strategy, where the same policy is evaluated from multiple permutation-invariant starting points. Instead of producing a single route, the decoder launches M parallel decoding trajectories. As a result, the model outputs a set of diverse candidate tours in a single forward pass: $\{\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(M)}\}$. This diversity arises naturally from the multi-start design rather than from stochastic sampling. Such a property is particularly beneficial for robust optimization, as multiple high-quality candidates can support downstream tasks including worst-case evaluation, recovery-based refinement, and robustness-guided selection.

These generated candidates then serve as inputs to the Robustness Estimator, which evaluates their fitness under uncertain environments.

4.3. Robustness Estimator

The Robustness Estimator module plays a central role in GRAPE by providing fast approximations of robustness-related quantities that would otherwise require solving additional optimization problems during training. In classical RCO, evaluating a candidate solution x under uncertainty often involves nested objectives such as $\max_{c \in \mathcal{U}} f(x, c)$, which can be computationally expensive when executed repeatedly inside learning loops.

To avoid these costs, GRAPE employs neural estimators that directly infer robustness values from the solution and problem parameters. These forward-efficient approximations eliminate the need for explicit inner adversarial optimization and enable scalable training over large instance batches.

The estimators serve two complementary roles within the framework:

- **Reward correction:** The Worst-Case Scenario (WCS) Estimator provides robustness-aware cost signals under perturbed environments to guide RL.
- **Computation simplification:** Estimators can replace expensive steps in other modules, ensuring differentiability and reducing per-iteration computation.

By replacing exact computations with trainable surrogate models, GRAPE maintains modularity and enables efficient optimization across different robust formulations.

4.3.1. Worst-case scenario Estimator

The WCS Estimator approximates the worst-case cost $\max_{c \in \mathcal{U}} f(x, c)$ for any candidate solution x . It is required by all robust formulations because evaluating the nested worst-case objective is the primary computational bottleneck.

We use lightweight neural networks as estimators:

$$\begin{aligned} E^{wcs} : \mathcal{NN}(x, \mathcal{P}_{\mathcal{U}}) &\approx \max_{c \in \mathcal{U}} f(x, c), \\ \mathcal{NN}_{ka}(\{x_1, \dots, x_k\}, \mathcal{P}_{\mathcal{U}}) &\approx \max_{c \in \mathcal{U}} \min_{i=1, \dots, k} f(x_i, c), \end{aligned} \quad (2)$$

where x is the solution, $\mathcal{U}$ is the uncertainty set, and $\mathcal{P}_{\mathcal{U}}$ is the problem-specific parameter describing $\mathcal{U}$.

This surrogate is trained with supervision from worst-case values on sampled solutions. Although generating these labels involves a non-differentiable sorting operation due to the combinatorial structure of the uncertainty set, the computation itself is straightforward. Specifically, under the budget uncertainty set, the worst-case scenario can be obtained by sorting all deviation ranges $\bar{c} - \underline{c}$ and greedily allocating the budget to the largest ones. We provide a theoretical justification for this approach in *Appendix B*.

In min-max, min-max regret and recoverable formulation, the estimator uses the single-solution form $\mathcal{NN}$. In k -adaptability formulation, the estimator adopts $\mathcal{NN}_{ka}$, where worst-case scenarios are identified by reformulating the problem into integer programming. Similar surrogate-based ideas have been employed in prior work such as Neur2BiLO [33], which shows that neural approximations can remain stable.

4.3.2. Regret Estimator

In regret-based formulations, the objective is to minimize the deviation between a chosen solution and the scenario-specific optimum:

$$\min_{x \in \mathcal{X}} \max_{c \in \mathcal{U}} \left[f(x, c) - \min_{x^* \in \mathcal{X}} f(x^*, c) \right]. \quad (3)$$

The inner term $\min_{x^* \in \mathcal{X}} f(x^*, c)$ corresponds to solving a deterministic CO problem $\mathcal{P}_c$ under the scenario whose cost is c , which is computationally expensive when evaluated repeatedly during training.

To avoid this costly step, we introduce a pre-trained DRL model as a surrogate solver. Given a solution x , the worst-case cost c is obtained within the same procedure as in the min-max setting. The surrogate solver then approximates the scenario-specific optimal value:

$$\text{Solver}(\mathcal{P}_c) = \min_{x^* \in \mathcal{X}} f(x^*, c). \quad (4)$$

This Solver shares the same architecture as the Generator module, but is trained offline on nominal instances and kept fixed during GRAPE training to ensure stability. It provides fast approximations of scenario-specific optimal values, enabling efficient computation of regret-based rewards without invoking exact optimization routines.

4.4. Solution Adjuster

In many two-stage robust formulations (e.g. recoverable RCO), decision-makers are allowed to adjust their solution within a limited range after the uncertain scenario is realized.

This setting naturally motivates the use of local search techniques to improve the initially proposed solution based on updated information. To this end, we introduce the Solution Adjuster module that helps recover the solution.

Specially, after obtaining the first-stage solution x from the Generator, we employ a 2-opt local search methods and solve $\min_{x' \in \mathcal{N}(x)} f(x', c)$, where $\mathcal{N}(x)$ denotes a local neighborhood of x , limited by a budget L . The neighborhood is defined by the L_1 distance over decision vectors: $\mathcal{N}(x) = \{x' \in \mathcal{X} \mid \|x' - x\|_1 \leq L\}$.

In practice, for routing tasks the neighborhood can be defined via simple 2-opt moves, while for more general CO problems it may consist of lightweight modifications such as item swaps, variable reassignments, or local re-allocations. Thus, although the Adjuster is illustrated with TSP-style operations, the mechanism is problem-agnostic and applies broadly across discrete optimization tasks that permit bounded second-stage adjustments.

Executing this local search process in every training iteration is computationally expensive. To alleviate this cost, we train a Recover Estimator to approximate the improvement gained through recovery. Formally, the estimator is defined as:

$$E^{rec} : \mathcal{NN}(x, \mathcal{P}_c) \approx f(x, c) - \min_{x' \in \mathcal{N}(x)} f(x', c). \quad (5)$$

This surrogate allows the framework to simulate the benefit of recovery actions without explicitly performing local optimization, thereby preserving training efficiency while supporting robust formulations that involve adjustable decisions.

4.5. Solution Picker

In k -adaptability formulations of RCO, the decision-maker selects k candidate solutions in the first stage and, after the uncertainty is revealed, chooses the best-performing one under the scenario. A solution set is considered robust if, for any possible scenario, at least one of the solutions performs well.

This motivates the need for a strategy to construct a set of solutions that are both high-quality and structurally diverse, so as to collectively defend against targeted adversarial scenarios. To achieve this, we propose a Solution Picker module that guides the Generator toward producing robust solution sets during training, inspired by principles from evolutionary computation and multi-objective optimization.

Given N candidate solutions (S_p) from the Generator, along with an additional set X_{ext} of externally generated solutions, the Picker constructs the final solution set S_k through four steps that combine robustness, regret, and diversity considerations:

1. **Nominal-optimal Selection:** Choose the solution x_n in the pool with the lowest cost under the nominal scenario (i.e., no uncertainty, using cost $\underline{c}$): $x_n = \arg \min_{x \in S_p} f(x, \underline{c})$. This captures the baseline performance of the instance without uncertainty.

2. Estimated robust-optimal Selection: Among the remaining candidates, select the one with the lowest estimated worst-case cost using the WCS Estimator: $x_w = \arg \min_{x \in S_p \setminus \{x_n\}} E^{wcs}(x)$. This encourages robustness against adversarial scenarios.

3. Regret-driven recovery solution: Compute the worst-case scenario for x_n , $c^{wcs}(x_n) = \arg \max_{c \in U} f(x_n, c)$. Then solve the problem under that scenario using the pre-trained DRL Solver: $x_r = \arg \min_{x \in \mathcal{X}} f(x, c^{wcs}(x_n))$. This solution is drawn from X_{ext} and serves as an adversarial complement.

4. Greedy quality-diversity pick: The remaining $k - 3$ solutions are picked from the pool S_p using a greedy scoring rule:

$$\text{Score}(x_i) = \alpha \cdot \text{Quality}(x_i, S) + (1 - \alpha) \cdot \text{Diversity}(x_i, S),$$

$$\text{Quality}(x_i, S) = \max_{c \in U} \min_{x \in \{x_i\} \cup S} f(x, c),$$

$$\text{Diversity}(x_i, S) = \frac{1}{|S|} \sum_{x \in S} \|x_i - x\|_1 \quad (6)$$

where S denotes the current partial selection. Here, *Quality* encourages performance complementarity in worst-case scenarios, while *Diversity* promotes structural dissimilarity to help hedge against different adversarial realizations. This balances solution quality and structural diversity within the set. In our experiments, the balance parameter in the scoring function is set to $\alpha = 0.1$, placing greater emphasis on solution diversity. This design encourages the Generator to produce solution sets that are not only strong individually, but also collectively resilient across a wide range of uncertain scenarios. To avoid duplicate selections, if a candidate has already been chosen in a previous step, the Picker selects the next-best solution instead.

Because computing *Quality* involves a nested max-min evaluation, we also replace it with a learnable Quality Estimator:

$$E^{qua} : \mathcal{NN}(x, S, \mathcal{P}_c) \approx \text{Quality}(x, S) \quad (7)$$

The final picked set $S_k = \{x_n, x_w, x_r, \dots\}$ via this scoring strategy is used to compute the reward. This module encourages the Generator to explore solution spaces that are not only individually robust but also collectively complementary. In brief, our Picker module design maintains a balance between structural robustness and solution optimality, enabling the learning policy to handle a wide range of uncertain scenarios effectively.

4.6. Training

Algorithm 1 summarizes the complete training process of our GRAPE framework. To learn robust policies, GRAPE adopts a RL framework in which the policy network interacts with an environment that exposes both nominal and uncertain cost information. At each iteration, the Generator produces multiple candidate solutions through multi-trajectory decoding. These trajectories naturally provide a diverse sample set for computing robust rewards.

Algorithm 1 Training Process of GRAPE Framework

Input: Problem instances S (cost matrix $\bar{c}, c$; uncertainty parameters Γ, k, L); Estimators $E(\cdot)$; POMO rollout number N ; Learning rate η ; Training steps T .

Output: Trained policy parameter θ

Initialize policy network π_θ ;

for $step \leftarrow 1$ to T **do**

$s^i \sim \text{Sample Instance}(S), \quad \forall i \in B$;

▷ 1. Solution Generation

$\{x_1, \dots, x_N\} \leftarrow \text{Multi-trajectory rollout}(s^i, \pi_\theta)$;

▷ 2. Robustness Estimation

$R(x_j^i) \leftarrow -E^{wcs}(x_j^i, s^i), \quad \forall j \in N, \forall i \in B$;

▷ 3a. Solution Regret (optional)

$R(x_j^i) \leftarrow R(x_j^i) + \text{Solver}(c), \quad \forall j \in N, \forall i \in B$;

▷ 3b. Solution Recover (optional)

$R(x_j^i) \leftarrow R(x_j^i) + E^{rec}(x_j^i, s^i), \quad \forall j \in N, \forall i \in B$;

▷ 3c. Solution Pick (optional)

$S_k \leftarrow \text{Pick from } \{x_1, \dots, x_N\} \cup X_{ext}$;

$R(x_j^i) \leftarrow -E^{wsc}(x_j^i, s^i), \quad \forall j \in S_k \cap X_{ext}, \forall i \in B$;

▷ 4. Policy gradient update

$b_i \leftarrow \frac{1}{N} \sum_{j=1}^N R(x_j^i), \quad \forall i \in B$;

$\nabla_\theta J(\theta) \leftarrow \frac{1}{BN} \sum_{i=1}^B \sum_{j=1}^N (R(x_j^i) - b_i) \nabla_\theta \log p_\theta(x_j^i)$;
 $\theta \leftarrow \theta + \eta \cdot \nabla_\theta J(\theta)$.

The policy is trained using REINFORCE with a baseline computed by averaging the rewards across the multi-start rollouts. To enhance generalization and exploration, GRAPE leverages data augmentation via cost matrix permutations. This aligns with the POMO multi-start mechanism by supporting exploration through structural permutations and exploitation of patterns that repeatedly yield strong robust performance.

During training, GRAPE unifies all robustness reasoning into environment's reward. The Generator outputs multiple nominal candidates, while the Estimators provide worst-case, regret, and recovery evaluations that replace explicit inner optimization. When required, the Solution Picker and the Solution Adjuster supply rewards for the corresponding problems. Although each module plays a distinct role, their interaction with uncertainty occurs only through reward construction, keeping the learning loop unified while supporting different robust formulations. Across formulations, GRAPE activates only the modules needed by each objective: min-max uses the Generator and WCS Estimator; regret additionally involves a fixed surrogate solver; k -adaptability uses the Picker to form a solution set; and recoverable applies the Adjuster.

5. Experiments

5.1. Experimental settings

We evaluate our GRAPE framework on all four RTSP formulations. Experiments are conducted on instances with $n = 20, 30, 40, 60$. Node coordinates are uniformly sampled

from $[0, 1]^2$. The lower bound matrix (nominal cost) $\underline{c}$ is computed as the Euclidean distances between nodes.

To introduce uncertainty, the upper bound matrix is defined as $\bar{c} = \underline{c} + s \cdot \Delta c$, where $\Delta c \sim \mathcal{U}[0, 1]$, and s (usually 1) is a deterioration scale that controls the degree of uncertainty. The budget Γ is set to $0.25n$ or $0.5n$. Setting Γ too large may reduce the problem to a deterministic one by neutralizing the effect of uncertainty (shown in *Appendix C*).

In the k -adaptability formulation, we fix k as 3 or 10. In the recoverable formulation, the recovery neighborhood size L is set to 6 or 10. These settings are consistent with prior work for fair comparison.

Although experiments primarily focus on routing tasks, GRAPE is designed to be problem-agnostic: only the Generator depends on task structure, while the Estimator and Adjuster remain unchanged. To verify its generality beyond RTSP, we additionally evaluate GRAPE on the Robust Knapsack Problem (RKP), where the instance construction and generator architecture differ slightly.

For RKP, each instance contains n items with values v_i and nominal weights w_i sampled from $[0, 1]$, and knapsack capacity set to $C = 0.3n$. Weight uncertainty follows $\bar{w}_i - w_i \sim \mathcal{U}[0, 1]$. All other settings remain consistent with the RTSP experiments. A set-based Generator is used in place of the sequential decoder in routing tasks, while the same Estimator and Adjuster apply, using 2-swap or greedy-repair neighborhoods instead of 2-opt.

All results are reported as averages over large sets of randomly generated test instances for each problem size. DRL-based methods, including GRAPE, are additionally evaluated over several independent runs to ensure statistical stability. Across all settings, the run-to-run variation for GRAPE is extremely small (standard deviation $< 0.1\%$), indicating highly stable performance.

5.1.1. Implementation details

All models are trained using a single NVIDIA GeForce RTX 3070 GPU with 32GB of memory. We follow most training settings from POMO. The encoder uses an embedding dimension of 128, and the feed-forward layers have hidden dimension 512. We use the Adam optimizer [34] with a learning rate of $\eta = 10^{-4}$ and weight decay $w = 10^{-6}$. Training proceeds for 500 epochs, each with 10,000 instances and a batch size of 256.

All neural estimators (e.g., E^{wcs} , E^{rec} , E^{qua}) share a common architecture with hidden dimension 256 and are trained separately via supervised learning on ground-truth labels. Labels are obtained from closed-form worst-case computations (for min-max), a fixed surrogate solver (for regret), or a small offline solver (for k -adaptability). Once pre-trained, all estimators are frozen during GRAPE training.

5.1.2. Baselines

1) Min-max formulation

- **Branch-and-Cut (BC)** is an exact method that combines duality-based reformulation with a

cutting-plane framework. It iteratively solves a master problem and adds cuts derived from the worst-case scenarios.

- **Genetic Algorithm (GA)** is a heuristic method implemented in PlatEMO [25]. We adapt it by incorporating an inner-loop procedure that searches for the worst-case scenario for each solution.
- **MO-RCO** [26] is a DRL-based method that addresses robustness by formulating the problem as a multi-objective optimization, balancing worst-case performance and nominal cost.

2) Min-max regret formulation (Regret)

- **Benders Decomposition (BD)** [23] is an exact method designed specifically for solving min-max regret TSP.
- **Simulated Annealing (SA)** [35] is a heuristic that uses local search combined with a simulated annealing acceptance criterion to explore the solution space.
- **Neural Combinatorial Optimization (NCO)** [14], based on MatNet, is a state-of-the-art DRL method for regret-based RCO.

3) k -adaptability formulation (K-Adapt)

- **Branch-and-Bound (BB)** [16] is an exact framework that systematically explores and prunes the solution space to handle the complexity of k -adaptability.
- **Column-and-Constraint Generation (CCG)** [36] is a method that iteratively solves a master problem and a pricing subproblem to generate candidate solutions for the problem.

4) Recoverable formulation (Recover)

- **Iterative Scenario (IS)** [20] is an exact method alternating between worst-case scenario identification and solving the recoverable RTSP until convergence.
- **Starting Solution and Local Search (SLS)** [20] is a heuristic that refines an initial recovered solution through constrained local search to efficiently improve solution quality.
- **Exact-ILP** [37] is an exact ILP-based approach for the recoverable RKP. It models weight uncertainty via Γ -scenario set and allows up to k item deletions after realization, using a compact $O(n^2)$ ILP that is solved to global optimality.

For larger-scale instances, we impose time limits on some algorithms: specifically, 3600 seconds for BB, BC and BD, and $15n$ seconds for SA.

Our model is experimented in both settings, with and without data augmentation $\times 8$.

Table 2

Results on different formulations of RCO problems (RTSP).

Problem / Method		$n = 20$			$n = 30$			$n = 40$			$n = 60$		
		Obj	Gap	Time/s	Obj	Gap	Time/s	Obj	Gap	Time/s	Obj	Gap	Time/s
Min-max	BC	7.24	0.00%	1.55	9.45	0.00%	28.4	11.23	0.00%	239	13.89	0.00%	3600
	GA	7.33	1.27%	3.74	9.77	3.43%	4.81	11.88	5.78%	5.61	15.57	12.11%	7.43
	MO-RCO	7.31	0.97%	0.37	9.56	1.16%	0.63	11.45	1.96%	0.73	14.50	4.39%	1.04
	GRAPE	7.26	0.28%	0.02	9.51	0.63%	0.03	11.34	0.98%	0.06	14.16	1.94%	0.15
	GRAPE ($\times 8$)	7.24	0.02%	0.02	9.49	0.42%	0.04	11.28	0.45%	0.17	14.10	1.51%	0.48
Regret	BD	2.09	0.00%	13.3	3.42	0.00%	41.7	4.21	0.00%	75.9	7.07	0.00%	3600
	SA	2.24	7.18%	300	3.70	8.19%	450	4.56	8.31%	600	7.78	10.04%	900
	NCO	2.11	0.96%	0.23	3.47	1.46%	0.31	4.29	1.90%	0.45	7.26	2.69%	1.76
	GRAPE	2.13	1.81%	0.07	3.51	2.53%	0.22	4.31	2.37%	0.34	7.24	2.40%	0.75
	GRAPE ($\times 8$)	2.09	0.04%	0.16	3.46	1.17%	0.41	4.24	0.71%	0.99	7.15	1.13%	2.96
K-Adapt	BB	5.58	0.00%	17.6	6.39	0.00%	63.1	7.36	0.00%	529	9.65	2.57%	3600
	CCG	5.66	1.43%	5.57	6.58	2.97%	9.95	7.64	3.80%	28.1	9.64	2.46%	239
	GRAPE	5.63	0.90%	1.17	6.57	2.81%	2.15	7.59	3.12%	4.53	9.49	0.82%	13.1
	GRAPE ($\times 8$)	5.63	0.81%	1.54	6.55	2.50%	3.60	7.56	2.72%	7.26	9.41	0.00%	29.0
Recover	IS	6.23	0.00%	22.7	7.52	0.00%	46.9	8.39	0.00%	87.8	9.72	0.00%	315
	SLS	6.31	1.29%	5.08	7.97	5.99%	12.4	9.77	16.47%	43.0	11.03	13.49%	146
	GRAPE	6.37	2.25%	0.09	7.75	3.06%	0.48	8.67	3.34%	1.33	10.15	4.43%	2.85
	GRAPE ($\times 8$)	6.30	1.12%	0.11	7.60	1.06%	0.76	8.52	1.55%	2.97	10.01	2.99%	7.14

5.1.3. Metrics

We evaluate all methods using three primary metrics: objective value, optimality gap, and running time. For the objective value, we compute the actual cost under the true worst-case realization, rather than relying on the surrogate predictions from the estimator used during training. This ensures a fair comparison across different methods.

5.2. Experimental results

5.2.1. Performance comparison

As shown in Table 2, our proposed GRAPE framework consistently achieves outstanding performance across all RCO formulations and problem sizes. In every type (including min-max, min-max regret, k-adaptability, and recoverable) of RTSP, GRAPE obtains objective values extremely close to those of exact methods, with gaps typically below 2%. At the same time, our framework significantly reduces computation time compared to those exact and heuristic baselines, especially as the problem size grows.

In addition to RTSP, we also report results on the RKP. As shown in Table 3, across all scales, GRAPE achieves: 1-4% lower worst-case cost than heuristic baselines and 3-10 $\times$ faster runtime due to amortized robustness estimation.

High solution quality and remarkable computational efficiency are the two main advantages of our work, which are attributed to the unified modular structure of GRAPE framework. By integrating solution generation, robustness estimation, and adaptive adjustment within an end-to-end

learning pipeline, GRAPE can efficiently capture the underlying structure of RCO problems and provide scalable, generalizable solutions.

5.2.2. Sensitivity analysis

We evaluate the sensitivity of GRAPE to variations in robustness parameters to verify its stability under different uncertainty levels.

From Table 4, we observe that GRAPE performs well across different problem settings. Varying the number of first-stage solutions k or the recovery size L leads to little performance loss, showing its adaptability to different robustness formulations.

From Table 5 we observe that even as the uncertainty budget Γ increases, which leads to a higher degree of uncertainty in the problem, GRAPE remains competitive compared to both exact and heuristic baselines.

These results highlight the flexibility of our framework in addressing diverse levels of uncertainty and limits of adjustment.

5.2.3. Generalization analysis

We further evaluate the generalization ability of GRAPE across both instance scales and uncertainty levels. As shown in Table 6, a model trained on smaller instances (e.g., $n = 20$) generalizes effectively to larger instances (e.g., $n = 40$), with minimal performance degradation.

Table 3

Results on different formulations of RCO problems (RKP).

Problem / Method		$n = 20$			$n = 40$			$n = 60$			$n = 100$		
		Obj	Gap	Time/s	Obj	Gap	Time/s	Obj	Gap	Time/s	Obj	Gap	Time/s
Max-min	BC	5.10	0.00%	1.20	10.17	0.00%	8.41	15.25	0.00%	31.9	25.34	0.00%	3600
	GA	5.02	1.57%	1.69	9.97	1.93%	2.85	14.88	2.48%	3.16	24.71	2.55%	6.20
	GRAPE	5.08	0.39%	0.02	10.10	0.63%	0.04	15.14	0.67%	0.10	25.21	0.51%	0.25
Regret	BD	1.47	0.00%	4.50	3.82	0.00%	35.1	7.79	0.00%	210	12.98	0.00%	3600
	SA	1.38	5.99%	300	3.58	6.37%	600	7.22	7.89%	900	12.15	6.85%	1500
	GRAPE	1.45	1.20%	0.05	3.77	1.22%	0.12	7.70	1.14%	0.25	12.87	0.86%	0.60
K-Adapt	BB	3.93	0.00%	6.34	6.68	0.00%	45.7	10.62	0.00%	245	17.72	0.00%	3600
	CCG	3.89	0.87%	4.33	6.62	0.86%	13.0	10.54	0.71%	42.5	17.61	0.65%	379
	GRAPE	3.92	0.20%	1.10	6.62	0.90%	2.13	10.54	0.79%	4.60	17.64	0.42%	11.3
Recover	ILP	4.39	0.00%	16.2	7.62	0.00%	89.6	10.71	0.00%	3600	17.85	0.00%	3600
	GRAPE	4.37	0.42%	0.07	7.54	1.05%	0.35	10.64	0.63%	0.95	17.78	0.37%	2.10

Table 4

Sensitivity analysis on uncertainty parameters (fixed $n = 20$), including varying solution number k and varying recovery size L . Each subtable compares GRAPE against applicable baselines (CCG cannot find a solution when $k = 3$).

a) k-adaptability			b) Recoverable		
Method	$k = 10$	$k = 3$	Method	$L = 6$	$L = 10$
BB	5.58	5.94	IS	6.23	6.13
CCG	5.66	—	SLS	6.31	6.27
GRAPE	5.63	5.97	GRAPE	6.37	6.19

Beyond scaling, we also assess adaptability to unseen uncertainty levels by varying the budget Γ during testing. When $\Gamma = 0$, the problem reduces to a nominal TSP, reflecting the model's ability to remain competitive under no uncertainty. Intermediate settings such as $\Gamma = 0.1n$ test sensitivity to mild perturbations, while larger budgets (e.g., $\Gamma = 0.5n$ or $0.75n$) correspond to strongly adversarial environments. Figure 3 demonstrates that GRAPE, despite being trained under a fixed uncertainty budget, remains more stable than other DRL methods when applied to both more optimistic and more pessimistic environments.

These results highlight GRAPE's generalization to unseen scenarios. Thanks to its modular architecture, the Generator and Robustness Estimator are designed as general-purpose components that adapt well to new environments.

5.2.4. Ablation analysis

We conduct ablation studies to quantify the impact of the Robustness Estimator on accuracy, efficiency, and final decision quality. As summarized in Table 7, the learned

Table 5

Sensitivity analysis on uncertainty budget Γ (fixed $n = 20$). GRAPE is compared against all baselines across formulations. The reported values are optimality gaps(%) w.r.t. the exact method.

Problem / Method		$\Gamma = 0.25n$	$\Gamma = 0.5n$	$\Gamma = 0.75n$
Min-max	GA	1.27	1.56	2.46
	MORCO	0.97	1.14	1.27
	GRAPE	0.28	0.51	0.44
Regret	SA	7.18	8.06	8.25
	NCO	0.96	2.14	4.30
	GRAPE	1.81	2.59	3.07
K-Adapt	CCG	1.43	2.54	6.51
	GRAPE	0.90	0.81	1.27
Recover	SLS	1.29	7.43	12.50
	GRAPE	2.25	3.11	4.44

WCS estimator provides highly accurate predictions, corresponding to at most 1% relative error in worst-case cost.

Replacing exact inner maximization with the estimator leads to substantial computational savings: training becomes roughly 600× faster, and inference is improved by over 20×, without compromising reward stability. Despite small approximation errors, the final robust objective differs by no more than 0.2% compared to using exact worst-case computation, indicating that policy optimization naturally absorbs estimator noise.

Overall, these results demonstrate that the estimator is both sufficiently accurate for decision-making and essential for scalability, enabling GRAPE to efficiently support all robust formulations without sacrificing solution quality.

Table 6

Generalization performance of GRAPE across different training and testing parameters for node size n and uncertainty budget Γ . BC denotes the exact baseline used for comparison.

a) On node size n					b) On budget Γ				
Test/Train	20	30	40	BC	Test/Train	5	10	15	BC
20	<u>7.26</u>	7.30	7.29	7.24	5	<u>7.26</u>	7.31	7.37	7.24
30	9.51	<u>9.50</u>	9.55	9.45	10	8.69	<u>8.68</u>	8.74	8.63
40	11.46	11.44	<u>11.34</u>	11.23	15	9.37	9.39	<u>9.35</u>	9.30

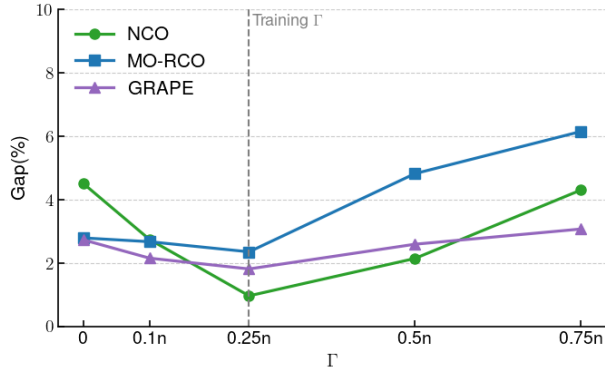


Figure 3: Generalization performance of DRL models across different values of Γ .

Table 7

Ablation analysis of the Robustness Estimator. Final Gap reports the difference in robust objective between models trained with vs. without the estimator.

n	Estimator Accuracy			Final Gap
	RMSE	R^2	MRE	
20	0.03	0.991	0.6%	0.1%
40	0.07	0.987	0.8%	0.2%
60	0.12	0.981	1.0%	0.2%

6. Conclusion

In this paper, we proposed GRAPE, a unified deep reinforcement learning framework for robust combinatorial optimization (RCO) under uncertainty. GRAPE addresses the limitations of existing RCO methods, which often rely on customized solvers for different robustness formulations. Specifically, the modular framework decomposes the problem into four components: Solution Generator, Robustness Estimator, Solution Picker, and Solution Adjuster. By adopting a data-driven prediction approach to approximate scenarios, regret, and adjustment, GRAPE avoids costly nested optimization while retaining high accuracy. Experiments demonstrate that GRAPE consistently outperforms existing

baselines in solution quality, computational efficiency, and generalization performance.

For future work, we plan to explore integrating advanced robustness estimators based on Transformer architectures [38] to further improve estimation accuracy and capture of RCO structure. Additionally, extending GRAPE to other combinatorial optimization problems beyond TSP and investigating theoretical guarantees on robustness and generalization remain promising directions.

CRedit authorship contribution statement

Yingze Zhong: Writing – original draft, Methodology, Visualization. **Hui Li:** Writing – review and editing, Conceptualization, Supervision. **Qinghang Zhai:** Data curation, Software, Writing – review and editing. **Zhan Shi:** Software. **Jianyang Sun:** Project administration.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledge

This work was partly supported by the National Natural Science Foundation of China (grant no.62072364, 62076197, 11991023), and National Key R&D Program of China under No. 2022YFA1004000.

Appendix A. Instantiation of RCO formulations for TSP and KP

In the main text, we use a generic objective $f(x, c)$ to represent the total cost (or negative profit) of a combinatorial solution x under uncertain parameters c . Below, we only specify the nominal objective and decision space for the two underlying CO problems used in our experiments.

TSP: Let $\mathcal{X}_{\text{tsp}} \subseteq \{0, 1\}^{|E|}$ denote all Hamiltonian tours on edge set E . With edge costs $c \in \mathbb{R}^{|E|}$, the nominal TSP objective is

$$f_{\text{tsp}}(x, c) = \sum_{e \in E} c_e x_e.$$

KP: For items with profits v_i , nominal weights w_i and capacity C , the feasible set is

$$\mathcal{X}_{\text{kp}} = \left\{ x \in \{0, 1\}^n \mid \sum_{i=1}^n w_i x_i \leq C \right\},$$

and the nominal objective, written in minimization form, is

$$f_{\text{kp}}(x, c) = - \sum_{i=1}^n v_i x_i.$$

Unlike RTSP, uncertainty in RKP affects the feasible set rather than the objective. Thus, the robust feasible set becomes

$$\mathcal{X}_{\text{rkp}} = \left\{ x \in \{0, 1\}^n \mid \max_{w \in \mathcal{U}} \sum_{i=1}^n w_i x_i \leq C \right\},$$

In our unified notation, this simply corresponds to treating the parameter vector as $c = (v, w)$. Therefore, both problems fit naturally into the same robust optimization framework used in GRAPE.

Appendix B. Discussions on the worst-case scenario under budget uncertainty

First, we prove that in a min-max formulation problem, the worst case scenario is specified when a solution x and the budget Γ are given, referred to the following lemma [39].

Lemma. *For a given solution x , the worst-case scenario $c^* \in \mathcal{U}$ for the min-max robust optimization problem*

$$\min_{x \in \mathcal{X}} \max_{c \in \mathcal{U}} \sum_{j \in N} c_j x_j$$

is achieved by setting

$$c_j^* = \bar{c}_j + \delta_j \xi_j^*,$$

where $\delta_j = \bar{c}_j - \underline{c}_j$ denotes the maximum deviation on edge j , and

$$\xi_j^* = \begin{cases} 1, & \text{for } j = j_1, j_2, \dots, j_\Gamma, \\ 0, & \text{otherwise,} \end{cases}$$

with $(j_1, j_2, \dots, j_n)$ being the indices sorted in non-increasing order of the values $\{\delta_j x_j\}$.

Proof: Consider any cost vector $c_j = \underline{c}_j + \delta_j \xi_j \in \mathcal{U}$, where $\xi_j \in [0, 1]$ and $\sum_{j \in N} |\xi_j| \leq \Gamma$. Then:

$$\begin{aligned} & \sum_{j \in N} c_j x_j - \sum_{j \in N} c_j^* x_j \\ &= \sum_{j \in N} (\underline{c}_j + \delta_j \xi_j) x_j - \sum_{j \in N} (\underline{c}_j + \delta_j \xi_j^*) x_j \\ &= \sum_{j \in N} \delta_j x_j (\xi_j - \xi_j^*) \\ &= \sum_{j \in N^+} \delta_j x_j (\xi_j - 1) + \sum_{j \in N^-, \xi_j < 0} \delta_j x_j \xi_j + \sum_{j \in N^-, \xi_j > 0} \delta_j x_j \xi_j, \end{aligned}$$

where $N^+ = \{j : \xi_j^* = 1\}$, $N^- = \{j : \xi_j^* = 0\}$.

Since ξ^* selects the Γ largest terms of $\delta_j x_j$, we can bound:

$$\begin{aligned} & \sum_{j \in N} \delta_j x_j (\xi_j - \xi_j^*) \\ & \leq \delta_{j_\Gamma} x_{j_\Gamma} \left(\sum_{j \in N^+} (\xi_j - 1) + \sum_{j \in N^-, \xi_j < 0} (-\xi_j) + \sum_{j \in N^-, \xi_j > 0} \xi_j \right) \\ & = \delta_{j_\Gamma} x_{j_\Gamma} \left(\sum_{j \in N} |\xi_j| - \Gamma \right) \\ & \leq 0, \end{aligned}$$

where the last inequality follows from the budget constraint $\sum_{j \in N} |\xi_j| \leq \Gamma$.

Therefore,

$$\sum_{j \in N} c_j x_j \leq \sum_{j \in N} c_j^* x_j,$$

which shows that c^* maximizes the objective under the given solution x , and hence is a worst-case realization. $\square$

This also applies to the min-max regret problem [14] and the recoverable problem.

According to Lemma, we immediately obtain the following corollary:

Corollary. *In a robust TSP, for any given tour, the worst-case cost is attained by selecting the Γ edges with the largest uncertainty ranges $(\bar{c}_j - \underline{c}_j)$ to take their upper bounds.*

Based on this result, our practical algorithm computes the worst-case cost by sorting the edges in a tour according to their uncertainty ranges and selecting the top- Γ ones.

This corollary captures a typical uncertainty structure in TSP problems. However, it does not generalize to all robust combinatorial problems. For example, in the robust job shop scheduling problem, the order of $\{\delta_j x_j\}$ must consider the global schedule structure, and the worst-case scenario cannot be determined by local sorting alone.

Finally, the k -adaptability formulation aims to minimize the worst-case objective value across the k solutions under any scenario within the uncertainty set $\mathcal{U}$. The corresponding worst-case cost is:

$$\max_{c \in \mathcal{U}} \min_{i=1, \dots, k} f(x_i, c)$$

To compute the worst-case objective under this model, we formulate the problem as a mixed-integer program (MIP). Let z denote the minimal cost among the k solutions under a scenario ξ . Then the following MIP computes the worst-case cost:

$$\begin{aligned} \max_{\xi \in \{0,1\}^n, z \in \mathbb{R}} \quad & z \\ \text{s.t.} \quad & z \leq f(x_i, \underline{c} + \delta \xi), \quad \forall i = 1, \dots, k \\ & \sum_{j=1}^d \xi_j \leq \Gamma \end{aligned}$$

Each inequality $z \leq f(x_i, \underline{c} + \delta \xi)$ ensures that z is no larger than the cost of each solution x_i under the current scenario.

In many applications (e.g., TSP), $f(x_i, c)$ is linear in c , and the formulation can be solved directly using MIP solvers such as Gurobi.

This MIP formulation provides an exact computation of the worst-case objective under the k -adaptability model, and is employed to generate ground-truth labels for training our estimators.

Appendix C. Clarification on the setting of budget Γ

In a RCO problem with budget uncertainty, budget Γ controls the total level of cost deviations permitted across all elements. Some literature suggests that in experimental settings, Γ should be set larger than $C(n, 2)$, or as $\lfloor \frac{C(n,2)}{4} \rfloor$, $\lfloor \frac{C(n,2)}{2} \rfloor$ [14].

We argue against such choices and explain the reasoning as follows.

Consider the min-max or min-max regret RTSP. Let $S \subseteq E$ denote the set of edges selected in a given tour x . Since any tour includes exactly n edges, the inner maximization problem (i.e., computing the worst-case scenario) becomes:

$$\begin{aligned} \max_{c \in \mathcal{U}} \quad & \sum_{(i,j) \in E} c_{ij} x_{ij} \\ = \quad & \max_{\xi \mid \sum_{(i,j) \in E} \xi_{ij} \leq \Gamma} \sum_{(i,j) \in E} \left[\underline{c}_{ij} + (\bar{c}_{ij} - \underline{c}_{ij}) \xi_{ij} \right] x_{ij} \\ = \quad & \max_{\xi \mid \sum_{(i,j) \in E} \xi_{ij} \leq \Gamma} \sum_{(i,j) \in S} \left[\underline{c}_{ij} + (\bar{c}_{ij} - \underline{c}_{ij}) \xi_{ij} \right] \\ \leq \quad & \max_{\xi \mid \sum_{(i,j) \in S} \xi_{ij} \leq \Gamma} \sum_{(i,j) \in S} \left[\underline{c}_{ij} + (\bar{c}_{ij} - \underline{c}_{ij}) \xi_{ij} \right] \\ = \quad & \sum_{(i,j) \in S} \bar{c}_{ij}. \end{aligned}$$

Conversely, we also observe that:

$$\begin{aligned} \max_{c \in \mathcal{U}} \quad & \sum_{(i,j) \in E} c_{ij} x_{ij} \\ = \quad & \max_{\xi \mid \sum_{(i,j) \in E} \xi_{ij} \leq \Gamma} \sum_{(i,j) \in S} \left[\underline{c}_{ij} + (\bar{c}_{ij} - \underline{c}_{ij}) \xi_{ij} \right] \\ \geq \quad & \max_{\xi \mid \sum_{(i,j) \in E} \xi_{ij} \leq \Gamma} \sum_{(i,j) \in S} \left[\underline{c}_{ij} + (\bar{c}_{ij} - \underline{c}_{ij}) \xi_{ij} \right] \\ = \quad & \sum_{(i,j) \in S} \bar{c}_{ij}. \end{aligned}$$

Combining both directions, we conclude that:

$$\max_{c \in \mathcal{U}} \sum_{(i,j) \in E} c_{ij} x_{ij} = \sum_{(i,j) \in S} \bar{c}_{ij}, \quad \text{whenever } \Gamma \geq n.$$

This implies that the RTSP reduces to a deterministic TSP whose cost is $\bar{c}$.

In summary, since any TSP tour includes only n edges, increasing Γ beyond n allows all selected edges to reach their upper bounds. The contributions from deviations on unselected edges (i.e., those in $E \setminus S$) do not affect the objective. Note that in our experiments, the smallest instance size is $n \geq 20$, for which $\lfloor \frac{C(n,2)}{4} \rfloor > n$ already holds. Therefore, excessively large values of Γ diminish the role of uncertainty and lead to trivialized robust counterparts.

References

- [1] I. Bello, H. Pham, Q. V. Le, M. Norouzi, S. Bengio, Neural combinatorial optimization with reinforcement learning, in: International Conference on Learning Representations (ICLR), 2017, pp. 1–15.
- [2] E. B. Khalil, H. Dai, Y. Zhang, B. Dilkina, L. Song, Learning combinatorial optimization algorithms over graphs, in: Advances in Neural Information Processing Systems, Vol. 30, 2017, pp. 1–11.
- [3] G. Infantes, S. Roussel, P. Pereira, A. Jacquet, E. Benazera, Learning to solve job shop scheduling under uncertainty, in: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2024), Vol. 14742 of Lecture Notes in Computer Science, Springer, Cham, 2024, pp. 370–386. doi:10.1007/978-3-031-60597-0_21.
- [4] H. Aissi, C. Bazgan, D. Vanderpooten, Min-max and min-max regret versions of combinatorial optimization problems: A survey, European Journal of Operational Research 197 (2) (2009) 427–438.
- [5] O. Vinyals, M. Fortunato, N. Jaitly, Pointer networks, in: Advances in Neural Information Processing Systems, Vol. 28, 2015, pp. 1–9.
- [6] M. Nazari, A. Oroojlooy, L. V. Snyder, M. Takáč, Reinforcement learning for solving the vehicle routing problem, in: Advances in Neural Information Processing Systems, Vol. 31, Curran Associates, Inc., 2018, pp. 9861–9871.
- [7] W. Kool, H. van Hoof, M. Welling, Attention, learn to solve routing problems!, in: International Conference on Learning Representations (ICLR), 2018, pp. 1–25.
- [8] Y.-D. Kwon, J. Choo, I. Yoon, M. Park, D. Park, Y. Gwon, Matrix encoding networks for neural combinatorial optimization, arXiv preprint arXiv:2106.11113 (2021).
- [9] V. Nair, S. Bartunov, F. Gimeno, I. von Glehn, P. Lichocki, I. Lobov, B. O'Donoghue, N. Sonnerat, C. Tjandraatmadja, P. Wang, R. Ad-danki, T. Hapuarachchi, T. Keck, J. Keeling, P. Kohli, I. Ktena, Y. Li, O. Vinyals, Y. Zwols, Solving mixed integer programs using neural networks, arXiv preprint arXiv:2012.13349 (2021).
- [10] J. Dumouchelle, E. Julien, J. Kurtz, E. B. Khalil, Neur2ro: Neural two-stage robust optimization, arXiv preprint arXiv:2310.04345 (2023).

- [11] E. Julien, K. Postek, I. Birbil, Machine learning for k-adaptability in two-stage robust optimization, *INFORMS Journal on Computing* 37 (3) (2024) 644–665.
- [12] M. Goerigk, J. Kurtz, Data-driven prediction of relevant scenarios for robust combinatorial optimization, *Computers & Operations Research* 174 (2025) 106886. doi:10.1016/j.cor.2024.106886.
- [13] P. Kouvelis, G. Yu, *Robust Discrete Optimization and Its Applications*, Vol. 14, Springer, 2013.
- [14] P. Xiao, Z. Zhang, J. Chen, J. Wang, Z. Zhang, Neural combinatorial optimization for robust routing problem with uncertain travel times, in: *Advances in Neural Information Processing Systems*, 2024.
- [15] G. A. Hanasusanto, D. Kuhn, W. Wiesemann, K-adaptability in two-stage robust binary programming, *Operations Research* 63 (4) (2015) 877–891.
- [16] C. Buchheim, J. Kurtz, Min–max–min robust combinatorial optimization, *Mathematical Programming* 163 (2017) 1–23.
- [17] A. Ben-Tal, A. Goryashko, E. Guslitzer, A. Nemirovski, Adjustable robust solutions of uncertain linear programs, *Mathematical Programming* 99 (2) (2004) 351–376.
- [18] C. Büsing, A. M. C. A. Koster, M. Kutschka, Recoverable robust knapsacks: the discrete scenario case, *Optimization Letters* 5 (2011) 379–392. doi:10.1007/s11590-011-0307-1.
- [19] M. Goerigk, A. Schöbel, Recovery-to-optimality: A new two-stage approach to robustness with an application to aperiodic timetabling, *Computers & Operations Research* 52 (2014) 1–15. doi:10.1016/j.cor.2014.06.004.
- [20] A. Chassein, M. Goerigk, On the recoverable robust traveling salesman problem, *Optimization Letters* 10 (2016) 1479–1492.
- [21] D. Bertsimas, M. Sim, The price of robustness, *Operations Research* 52 (1) (2004) 35–53.
- [22] A. Ben-Tal, A. Nemirovski, Robust convex optimization, *Mathematics of Operations Research* 23 (4) (1998) 769–805.
- [23] R. Montemanni, J. Barta, M. Mastrolilli, L. M. Gambardella, The robust traveling salesman problem with interval data, *Transportation Science* 41 (3) (2007) 366–381.
- [24] J. Zhang, W. Wu, M. Yagiura, A heuristic approach for the robust traveling salesman problem, in: *2017 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*, 2017.
- [25] Y. Tian, R. Cheng, X. Zhang, Y. Jin, Platemo: A matlab platform for evolutionary multi-objective optimization, *IEEE Comput. Intell. Mag.* 12 (4) (2017) 73–87.
- [26] Y. Zhong, H. Li, Z. Shi, J. Sun, Deep reinforcement learning model for robust travel salesman problem via multiobjective optimization framework, in: *2025 IEEE Congress on Evolutionary Computation (CEC)*, 2025, pp. 1–4. doi:10.1109/CEC65147.2025.11043069.
- [27] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, S. Min, Pomo: Policy optimization with multiple optima for reinforcement learning, in: *Advances in Neural Information Processing Systems*, Vol. 33, Curran Associates, Inc., 2020, pp. 21188–21198.
- [28] W. Pan, H. Xiong, J. Ma, W. Zhao, Y. Li, J. Yan, UniCO: On unified combinatorial optimization via problem reduction to matrix-encoded general tsp, in: *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [29] Z. Liao, J. Chen, D. Wang, Z. Zhang, J. Wang, BOPO: Neural combinatorial optimization via best-anchored and objective-guided preference optimization, in: *Proceedings of the Forty-Second International Conference on Machine Learning (ICML)*, 2025.
- [30] C. Zhou, C. Yu, S. Yao, X. Lin, Z. Wang, Y. Zhou, Q. Zhang, Urs: A unified neural routing solver for cross-problem zero-shot generalization, *arXiv preprint arXiv:2509.23413* (2025). arXiv:2509.23413.
- [31] Y. Shui, H. Li, J. Sun, Q. Zhang, Approximating robust pareto fronts by the meof-based multiobjective evolutionary algorithm with two-level surrogate models, *Information Sciences* 657 (2024) 119946. doi:https://doi.org/10.1016/j.ins.2023.119946.
- [32] J. Deng, J. Sun, Q. Zhang, H. Li, Expected hypervolume improvement is a particular hypervolume improvement, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, 2025, pp. 16217–16225.
- [33] J. Dumouchelle, E. Julien, J. Kurtz, E. B. Khalil, Neur2bilo: Neural bilevel optimization, *arXiv preprint arXiv:2402.02552* (2024).
- [34] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, *arXiv preprint arXiv:1412.6980* (2014).
- [35] C.-C. Lu, S.-W. Lin, K.-C. Ying, Minimizing worst-case regret of makespan on a single machine with uncertain processing and setup times, *Applied Soft Computing* 23 (2014) 144–151.
- [36] J. Kurtz, Approximation algorithms for min-max-min robust optimization and k-adaptability under objective uncertainty, *arXiv preprint arXiv:2106.03107* (2023).
- [37] C. Büsing, A. M. C. A. Koster, M. Kutschka, Recoverable robust knapsacks: γ -scenarios, in: *International Network Optimization Conference (INOC)*, Vol. 6701 of LNCS, Springer, 2011, pp. 583–588.
- [38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: *Advances in Neural Information Processing Systems*, Vol. 30, Curran Associates, Inc., 2017, pp. 5998–6008.
- [39] K. Hasegawa, W. Wu, A heuristic approach for the robust traveling salesman problem, in: *2022 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*, 2022, pp. 561–565.